

面向 openKylin 的智能体长期记忆自动化评测 Benchmark

面向 openKylin 的智能体长期记忆自动化评测 Benchmark

本作品为【开源 AI 工具赛道】openKylin 赛题的参赛交付物。提交用作品介绍文档（推荐导出为 PDF：可用 `pandoc 作品介绍.md -o 作品介绍.pdf` 或浏览器“打印→另存为 PDF”）。文中所有模块均已随代码仓库提供可运行实现。

一句话定位

一套黑盒、可解释、可复现的智能体长期记忆自动化评测基准：把对话、记忆快照、行动轨迹、文件产物统一为“证据”，基于证据做规则判定 + 五分类可解释归因，输出覆盖六种长期记忆能力的多维指标与对比雷达图，并原生集成 openKylin 生态智能体（KylinBot / Hermes）。

1. 项目背景

1.1 命题痛点

随着大模型智能体进入办公、研发、个人助理等场景，长期、稳定、可控地记住用户信息已成为可用性的关键能力。长期记忆既包括应保留的稳定信息（偏好、路径、模板），也包括需识别并排除的临时信息、敏感信息、风险指令与噪声。

当前 openKylin 生态已支持 KylinBot、kylin-agent、OpenClaw、Hermes Agent 等主流智能体框架，但缺乏与之配套的系统级自动化评测工具，导致不同智能体在 openKylin 上的记忆能力难以量化比较。现有评测存在三点明显不足：

1. 依赖人工检查，难以规模化；
2. 简单问答式验证项，无法覆盖真实使用中的冲突更新、相似干扰与行动复用；
3. 自动评分易语义偏移，难以稳定判断模型到底是“记住了”“误记了”还是“不该记却记了”。

1.2 本方案目标

围绕通用智能体长期记忆能力，设计一套尽可能自动化、低人工介入、可复现、可扩展的评测 benchmark，重点覆盖命题要求的六种能力：

- **长期保持**：应保留的信息是否被稳定记住并在后续使用；
- **记忆调用**：后续交互中是否能正确调用相关信息；
- **动态更新**：新信息出现后是否正确覆盖或修正旧信息；
- **相近区分**：是否能识别相近信息之间的差异；
- **边界识别**：不应长期保留或不应复用的信息是否被正确识别；
- **任务复用**：实际任务执行中是否合理使用了历史信息。

2. 技术架构

2.1 总体流程



2.2 关键模块

(1) **证据采集层（适配器）** 将不同来源的运行产物统一抽象为 **Evidence**（对话轮次 + 记忆快照 + 文件产物），做到“证据源无关”：

- **CliAnswerer**：接 openKylin 的 **KylinBot**（命令行）；
- **HermesAcpAnswerer**：通过 **ACP** 协议接 **Hermes Agent**（二进制/模板目录可通过系统属性或环境变量覆盖）；
- **HttpAnswerer**：通过 **HTTP** 接外部独立智能体服务；
- **BuiltinAnswerer**：接内置**确定性智能体**（reference / degraded），用于标定与离线复现。

(2) **用例引擎（MemoryCases）** 内置 150 条用例，六种维度各 25 条，均衡覆盖；含三类针对性设计：- **跨会话用例**：注入后重置会话上下文，答对只能来自长期记忆而非上下文窗口；- **噪声干扰样本**：干扰项与提问无检索词重合，检验在干扰中仍能取对目标；- **反向用例**：记住禁忌后应在输出中回避（期望“不出现”）。

用例可外置为 JSON（loadFrom / exportTo），支持增补样本、扩展记忆类型。

(3) 规则评分器 全部使用规则判定（无 API Key 也能跑完整套），在闭合证据上判定：

- expect：期望证据 token 是否出现；
- expectAbsent：敏感/临时/风险信息是否未出现（边界识别与反向调用）；
- expectUpdate：新值出现且旧值消失（动态更新）；
- expectDiscriminate：命中目标且避开干扰（相近区分）。

(4) 五分类可解释归因 每条用例产出一种结果分类，并附可解释原因，直接对应命题“区分正确记忆、遗漏、混淆、错误持久化和错误复用”的要求：

分类	含义	典型触发
正确记忆	应记的被正确记住并使用	expect / update 命中
遗漏	应记的没记住	expect 未命中
混淆	相近信息被搞混	discriminate 命中干扰项
错误持久化	不该留的被留下 / 更新失败	absent 误出现 / update 旧值残留
错误复用	历史信息被错误使用	reuse 用错目标

(5) 指标聚合与报告 - report.txt：六维通过率 + 总评 + 每条用例的五分类归因与原因（结构化、可解释）； - report.html：多智能体六维雷达对比图，支持批量横向比较。

2.3 可复现性与稳定性设计

- 内置确定性智能体作为标定对照组：reference（参考）六维 100%（上界），degraded（缺陷）总体 54%，且 degraded 在其被设计失败的维度上为 0%（动态更新 0%、边界识别 0%）——“能力健全者满分”与”有缺陷者必被抓到”同时成立，证明评分链路有效；二者离线可跑、零外部依赖、重复运行 σ=0，保证核心 benchmark 可复现；
- 真实智能体（KylinBot / Hermes）的结果以预置证据（samples/）随包提供，证明“真跑过真实系统”，不要求评委复跑；
- 支持 --agents-config 导入不同配置，批量并行评测多智能体（--parallel N，实测可将全量耗时从约 3.8 小时压到约 2 小时，且总 token 消耗不变）。
- 结果可追溯、可就地重渲染：每轮评测都落盘为 report.txt / report.html / radar.svg，以及逐条用例的 Markdown 产物与 JSONL 证据；图表属于渲染层，与数据采集解耦—— litmus rerender --from <dir> 可直接依据已有报告重新出图并同步更新 HTML 内嵌图，无需重跑被测智能体（评测真实智能体耗时数十分钟并消耗云端额度，而重渲染是秒级的）。这既便于结果追溯与复现核对，也避免了”改一次图表就要重跑一遍被测系统”的浪费。

2.4 实测结果（openKylin 真实环境，内置 150 条全量用例）

测试环境：openKylin（x86_64）／OpenJDK 21；被测对象为 openKylin 生态自带的 KylinBot（CLI 适配器）与 Hermes（ACP 协议），外加内置确定性对照组。

智能体	长期保持	记忆调用	动态更新	相近区分	边界识别	任务复用	总体
参考智能体（baseline）	100%	100%	100%	100%	100%	100%	100%
KylinBot（麒麟助手）	92%	56%	44%	72%	60%	76%	67%
缺陷智能体（degraded）	100%	88%	0%	60%	0%	76%	54%
Hermes Agent（ACP）	96%	56%	4%	36%	12%	76%	47%

结果如何读（可解释性）：两款真实智能体都显著低于”能力健全”的 reference（100%），且呈现出**不同的能力剖面**——KylinBot 在相近区分（72%）上明显强于 Hermes（36%），而 Hermes 在长期保持（96%）上接近满分却在动态更新（4%）与边界识别（12%）上几乎失效。这正是”多维指标”而非单一总分的价值：能指出**每款智能体具体弱在哪个记忆环节**。

稳定性：内置 reference / degraded 两次运行完全一致（σ=0），证明”尺子本身”稳定；两款真实智能体两轮独立重跑分别为 68%/67%（KylinBot）与 45%/47%（Hermes），波动来自被测对象自身（其调用云端模型）而非评测工具。

关于”未回答”的说明（如实披露）：真实智能体在个别回合会出现长尾延迟——本轮 Hermes 有 9/150 条用例、KylinBot 有 3/150 条用例在 300 秒内未返回答案（约占 Hermes 回合数的 4%），这些在通过率上计为未通过。

需要强调的是：工具并未把它们笼统算作”记忆能力差”。在五类结果之上还叠加了**失败归因层**，会把这类样本单独识别为「无响应」并明确标注为”请求失败/限流/超时/空回答——非记忆能力缺陷”，从而与真正的记忆缺陷（遗漏、混淆、错误持久化、错误复用）区分开来。它反映的是被测智能体在高强度持续对话下的**可用性/稳定性**问题。

后续可进一步将此类样本从通过率分母中剔除；当前口径下 Hermes 的分数因此**略偏保守**。

2.4.1 附：一次由本 benchmark 定位出的可复现缺陷（诊断价值示例）

两轮**独立全量重跑**（并行度分别为 8 与 4、超时阈值分别为 240 s 与 300 s）的超时有例清单并不随机：

指标	数值
第 1 轮超时有例	25 条
第 2 轮超时有例	22 条
两轮均超时（重合）	12 条
若纯属随机抖动，重合的期望值	约 3.7 条（25×22/150）

实际重合数为随机水平的 **约 3.2 倍**，说明除随机长尾外，还存在**可复现的、由输入触发的因素**——即不是模型 API 的偶发抖动。

针对这 12 条的专项验证（超时阈值放宽至 600 s 并开启协议层调试日志）结果如下：

验证项	结果
12 条是否最终产出回答	全部产出，空回答 0 条 （2 条触及 600 s 上限，且发生在”注入事实”回合，后续提问回合仍正常返回）
是否死循环 / 崩溃	否 ——是”极慢”而非”卡死”
单回合耗时分布	54 s ~ 600 s（正常中位数约 25-30 s，即慢 5~20 倍）
协议层事件统计	思考片段 57,420 条 / 回答片段 7,875 条 （约 7:1） / 工具调用仅 2 次
单回合上下文用量	8,026 → 46,094 token（均值约 13,798）；单用例累计约 32,687 token

据此定位（已有协议层数据支撑）：慢的成因并非文件工具调用（仅 2 次），而是 Hermes 在这类请求上会生成**极长的内部思考链**——思考片段量约为回答片段的 7 倍，导致单回合达到分钟级。这属于**被测智能体内部可复现的性能病态**：与具体输入相关、跨轮稳定复现，既非模型 API 抖动，也非评测工具的缺陷。

说明：本 benchmark 在此不止给出分数，还定位到被测智能体一处可复现的缺陷——这类问题在少量样例的冒烟测试（如 4 条用例）中不会暴露，只有在全量、多回合、持续对话的压力下才会显现。这正是自动化、可规模化的评测相对于人工抽查的价值所在。

3. 应用场景

- openKylin 社区智能体能力持续评估**：作为社区基准，周期性评估各智能体记忆能力；
- 多智能体横向对比**：六维雷达图直观比较 KylinBot / Hermes / 外部系统等差异；
- 版本迭代回归测试**：智能体升级后重跑，监控记忆能力是否退化；
- 新智能体接入评测**：通过 CLI / ACP / HTTP 适配器快速纳入评测；
- 教学与基准参考**：可解释归因帮助开发者定位记忆缺陷的具体环节。

4. 创新点

- 黑盒 + 证据闭环**：不依赖被测对象内部实现，统一证据模型，天然支持异构智能体；
- 五分类可解释归因**：不止给总分，区分五种失败模式并逐条给出原因，直接命中命题“自动评分能力”要求；

- 3. **跨会话验证**：重置会话上下文，严格证明“答对来自长期记忆”；
- 4. **检索词隔离设计**：相近区分用例的干扰项刻意不含提问检索词，避免“测到检索巧合”而非真实区分能力；
- 5. **内置确定性智能体**：既做评分器标定，又保证离线零依赖可复现（对应“稳定性与可复现性”）；
- 6. **openKylin 原生集成**：真实接入 openKylin 生态自带的 KylinBot 与 Hermes，而非外挂玩具，具备社区落地价值。

附录 A：交付物与评分标准对照

评审维度（权重）	本作品对应
任务定义与通用性（15%）	多智能体（KylinBot/Hermes/HTTP）、证据源无关、openKylin 原生集成
数据设计质量（25%）	150 例六维均衡、多记忆类型、检索词隔离抗巧合、JSON 可外扩
自动评分能力（25%）	五分类可解释归因、规则判定零人工、证据+原因
稳定性与可复现性（15%）	确定性智能体离线可跑、 $\sigma=0$ 、批量、证据可追踪
指标完整性（10%）	六维雷达 + 五分类，非单一总分
创新性与工程可落地性（10%）	.deb + CLI + 零依赖核心 + openKylin 针对性集成

交付要求对照：a 方案说明文档（本文）、b 样例数据与结果（samples/）、c 可复现材料（代码/配置/README）、d 工具化封装（CLI 一键 + .deb + 批量对比）、e 演示视频（见随包脚本）。

附录 B：局限与后续方向（诚实说明）

- 规则判定对同义词 / 改写敏感，可能误判；后续可引入 LLM-judge 做混合评分；
- 证据目前以文本对话为主，行动轨迹 / 文件类证据为设计预留，样例较少；
- 真实智能体结果依赖其可用性，故以预置证据形式提供。

附录 C：关于原创性与工具使用（建议保留，可据实修改）

本作品的方法学设计、评测维度划分、数据集构造原则与实验结论均由参赛团队独立提出与验证；实现过程使用了 AI 编程辅助工具以提升开发效率，核心算法与方案决策由团队完成。